SWE Roadmap: Backend, AI and Dynamics (Living Checklist)

Talha Munir Saeed

Built from my degree, my projects and my internship. Last updated 11 Oct 2026. Tick items off as I go (change `[ ]` to `[x]`).

How to use this roadmap

- Phases are in dependency order, but several run in parallel (marked). Week ranges assume about 5 to 6 focused hours a day; stretch them if I have less time.
- Every phase ends with something shipped (a repo, a deployed app, a write-up). Reading without shipping does not stick, and a shipped project is what goes on the resume.
- Start applying around Milestone 3, not after finishing everything. Interviews will show what to learn next.
- Keep a daily log: what I studied, what I built, what confused me.

Where I stand

Strong: DSA and algorithms (4.0), AI (4.0), PDC (4.0), FYP (4.0), numerical computing (4.0), plus theory in deep learning, networks, databases, SE, security and compilers.

Weak spots to fix first: operating systems (1.67) and information security (2.67). OS is a common interview topic, so it goes high on the revision list.

Main gap: real backend engineering: REST APIs, auth, Postgres/Mongo, Docker, cloud, CI/CD, system design and deployed projects. Also React and production-style AI apps.

Internship base: dev/staging/prod environments, JS/TS, Express validators, Postman testing, serverless CI/CD theory, time zones, Stripe docs, code documentation.

The strategy in one page

- The spine is backend. AI engineering, DevOps and Dynamics all hang off it.
- Two main languages: Python (AI, scripting, FastAPI) and TypeScript (Node backend, React, React Native). Keep C/C++ warm only for OS and DSA understanding. Treat SQL as a real language.
- Add C# later: X++ reads like C#, and .NET is common in the Dynamics ecosystem.
- Do not collect ten languages. After two are solid, a third takes weeks, not months.
- Dynamics and Power Platform run as a low-intensity parallel track (3 to 5 hours a week) so they are ready when I want the consultancy route.

Phase 0: Setup (week 1)

☐ GitHub profile cleaned, with an intro README; push all old projects (Zakat calculator, fantasy football, SMTP server, etc.) with proper READMEs

- [ ] Install: WSL2 or Linux, Git, Docker Desktop, VS Code, Python (uv or venv), Node LTS, Postman
- [ ] Create a LeetCode/NeetCode account; update LinkedIn
- [ ] Start the daily log

Phase 1: Language foundations (weeks 1-5)

Python

- [ ] Core: types, comprehensions, generators, decorators, context managers, dataclasses, type hints, error handling
- [ ] OOP and modules, virtual environments, packaging, pytest
- [ ] Async (asyncio), HTTP clients (httpx), file and JSON handling
- [ ] CS50 Python alongside
- [ ] Mini project: CLI expense tracker with tests

TypeScript / JavaScript

- [ ] Modern JS: let/const, arrow functions, destructuring, modules, promises, async/await, the event loop
- [ ] TypeScript: types, interfaces, generics, narrowing, utility types, tsconfig
- [ ] Node.js: fs, http, npm, environment variables
- [ ] Mini project: typed CLI or small Express API

SQL

- [ ] SELECT, JOIN, GROUP BY, subqueries, window functions, CTEs; practise on SQLZoo or LeetCode SQL

Bash

- [ ] Variables, loops, pipes, grep/sed/awk, cron, file permissions

C/C++ upkeep

- [ ] 30 minutes a week on pointers, memory and struct layout; it feeds OS revision

Phase 2: CS core in interview mode (weeks 2-12, parallel, 1-2 h/day)

DSA (4.0 in the course, so this is re-sharpening, not relearning)

- [ ] Redo the patterns in Python: arrays/hashing, two pointers, sliding window, stack, binary search, linked lists, trees, heaps, tries, graphs (BFS/DFS/topological sort/Dijkstra/union-find), backtracking, DP, greedy, intervals
- [ ] Target: NeetCode 150, then timed mock interviews
- [ ] Keep a notebook: pattern, trigger, template

Operating systems (priority, grade 1.67)

- [ ] OSTEP (free book): virtualization (processes, scheduling, memory, paging), concurrency (locks, condition variables, deadlock), persistence (file systems)
- [ ] Hands-on: write a tiny shell in C, a thread pool, and give the scheduler visualizer a README with screenshots
- [ ] Be able to explain: process vs thread, context switch, virtual memory, race conditions, mutex vs semaphore, deadlock conditions

Databases

- [ ] ACID, isolation levels, B-tree indexes, query plans (EXPLAIN), normalization, locking

Networking

- [ ] HTTP/1.1 vs 2, TLS, DNS, TCP handshake and congestion basics, sockets in Python; connect this to the SMTP project

OOP and design

- [ ] SOLID, common design patterns (factory, strategy, observer, singleton), clean code

Security (grade 2.67)

- [ ] OWASP Top 10, hashing vs encryption, password storage, JWT/OAuth2/OIDC, CSRF/XSS/SQL injection and how to prevent them

Math

- [ ] A weekly slot (I enjoy it): linear algebra and probability for ML, gradient descent, Bayes

Phase 3: Backend engineering (weeks 4-16), the biggest gap

3.1 API fundamentals

- [ ] REST design, status codes, idempotency, pagination, filtering, versioning, OpenAPI/Swagger

3.2 Frameworks

- [ ] FastAPI (Python) first, then Express or NestJS (TypeScript). Build the same API in both

3.2b Express in depth (TypeScript)

- [ ] Setup: Express with TypeScript (tsx or ts-node-dev, tsconfig), a layout of routes, controllers, services and middleware, and environment config validated with Zod
- [ ] Routing: routers, route params, query strings, request bodies, versioned routes (/api/v1)
- [ ] Middleware: order matters. Built-in (json, urlencoded), third-party (cors, helmet, morgan or pino-http) and custom (request ID, auth, logging)
- [ ] Error handling: async errors, one central error-handling middleware, a consistent error response shape, correct status codes

- [ ] Validation: express-validator (already used at the internship) and Zod; validate body, params and query at the edge
- [ ] Auth: JWT access and refresh tokens, password hashing, role checks as middleware
- [ ] Data: Postgres with Prisma or Drizzle, MongoDB with Mongoose, transactions and migrations
- [ ] Uploads and limits: multer with S3-compatible storage, express-rate-limit, CORS configuration, helmet security headers
- [ ] Testing: Jest or Vitest with supertest, plus Postman collections
- [ ] Documentation: generate OpenAPI/Swagger from the routes
- [ ] Compare with NestJS: the same API using modules, controllers, providers and decorators, and know when that structure helps
- [ ] **Project A2:** build the Pocket Plan API a second time in Express and TypeScript, then compare it with the FastAPI version (speed to write, type safety, tooling, performance)

3.3 Databases

- [ ] PostgreSQL as primary: schema design, migrations, indexes; an ORM (SQLAlchemy or Prisma)
- [ ] MongoDB: documents, when to choose it over SQL, aggregation basics
- [ ] Redis: caching, sessions, rate limiting, simple queues

3.4 Auth and security

- [ ] Sessions vs JWT, refresh tokens, OAuth2 login (Google), RBAC, password hashing (bcrypt/argon2), secrets in env vars or a secret manager

3.5 Quality

- [ ] Validation (Pydantic and Zod), error handling, structured logging
- [ ] Testing: pytest, Jest/Vitest, integration tests with a test database, Postman/Newman collections

3.6 Async work and integrations

- [ ] Background jobs (Celery or BullMQ), queues (start with Redis/SQS, understand Kafka concepts), webhooks
- [ ] Stripe in test mode: Checkout, payment intents, subscriptions, webhooks with signature verification, idempotency keys
- [ ] Email (SMTP/SendGrid), file uploads (S3-compatible storage), third-party API keys handled safely, rate limiting, time zones (store UTC)

3.7 Shipping

- [ ] Docker and docker compose, environment management, GitHub Actions CI (lint, test, build), deploy to Render/Railway/Fly and then a major cloud, one serverless function (Lambda or Azure Functions)
- [ ] Health checks, logs, basic metrics

3.8 Stack tour, middleware and tooling

- [ ] Understand the common stacks and their trade-offs: MERN (MongoDB, Express, React, Node), MEAN (Angular instead of React), PERN (Postgres instead of Mongo), T3 (Next.js, TypeScript, tRPC, Prisma, Tailwind), Java (Spring Boot), Python (FastAPI or Django), .NET (ASP.NET Core). Build one small CRUD app in two of them and read about the rest
- [ ] Request types in depth: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS, plus headers, cookies, CORS, content types and status code families
- [ ] Middleware: logging, auth, validation, error handling, CORS, rate limiting, request IDs, in both Express and FastAPI
- [ ] Tooling: pip, uv, venv, npm, dotenv with a .env.example, nodemon or tsx watch, ESLint, Prettier, Postman or Bruno
- [ ] ACID in practice: write a transaction that fails halfway and show the rollback; reproduce dirty reads, non-repeatable reads and phantom reads at different isolation levels
- [ ] Cloud computing basics: IaaS vs PaaS vs serverless, regions, VPC, IAM, object storage, managed databases, and billing alerts so a free tier never becomes a bill
- [ ] Deploy with Docker end to end: Dockerfile, compose, image registry, secrets, reverse proxy (Nginx or Caddy), HTTPS
- [ ] Scale thinking: what changes between 100 and 1 million requests per second (caching, read replicas, queues, horizontal scaling, CDN, load shedding). Practise these as paper designs in Phase 5

3.9 Planning documents to write before coding

Keep each one short and write them before every real project. They double as resume and interview material, and they are exactly what consultancy work asks for.

- [ ] PRD: problem, users, goals, non-goals, success metrics, scope for v1
- [ ] User stories with acceptance criteria, grouped into themes and epics, then prioritised
- [ ] User flows and wireframes (Figma or paper) before any UI code
- [ ] Design system: colors, type scale, spacing, core components, so the UI stays consistent
- [ ] Architecture doc: component diagram, data model (ERD), API list, deployment diagram, key trade-offs. Reuse UML skills from the software design course
- [ ] SRS (functional and non-functional requirements) and SDS (design specification) for at least one project, to practise the formal versions
- [ ] WBS, task breakdown and estimates, tracked as tickets in GitHub Projects or Jira

- [ ] Test plan: what gets unit, integration and end-to-end tests, plus acceptance tests taken from the user stories
- [ ] For AI features: an agent spec (goal, tools, inputs and outputs, failure modes), an eval set, and a prompt version log
- [ ] Use a light template for each, so planning a small project takes an hour, not a week

3.10 Project structure and clean code

- [ ] Backend layouts: layered (routes, services, repositories) and feature-based (a folder per feature); when each fits, and why small projects stay flat
- [ ] Frontend structure: feature-based folders, atomic design (atoms, molecules, organisms, templates, pages), tests and styles next to their components
- [ ] Domain-driven design basics: entities, value objects, aggregates, repositories, bounded contexts, and when it is overkill
- [ ] SOLID applied to real code, plus dependency injection, interfaces over concrete classes, small functions, clear naming
- [ ] Maintainable-code habits: linting, formatting, type checking, a code review checklist, clear commit messages, short architecture decision records
- [ ] Reference layouts for the AI projects: a RAG backend (ingestion, chunking, embeddings, retrieval, generation and evals as separate modules) and an agent backend (tools, prompts, orchestration, memory, tracing)

Project A: Pocket Plan API (flagship backend). Rebuild Pocket Plan as a real service: auth, Postgres, budgets/goals/transactions, a Stripe test subscription, an AI categorization endpoint, tests, Docker, CI, deployed, with OpenAPI docs and a README containing an architecture diagram.

Phase 4: Frontend and mobile, good enough (weeks 8-18, light, parallel)

Goal: read, modify and assemble components confidently and build working dashboards. Not become a design specialist.

- [ ] HTML/CSS: semantic HTML, flexbox, grid, responsive layouts
- [ ] Browser JS: DOM, fetch, events
- [ ] React: components, props, state, hooks (useState, useEffect, useMemo), routing, forms, data fetching (TanStack Query)
- [ ] Tailwind plus a component library (shadcn/ui): copy a component and understand every line
- [ ] Next.js basics: routing, server vs client components, API routes
- [ ] Core principles: state management, accessibility basics, performance basics, auth on the frontend
- [ ] Mobile: React Native with Expo (reuses React and TypeScript). Flutter only if a job requires it

UI design styles and layout sources

- [ ] Learn the main visual styles and when each fits: minimal/flat, glassmorphism, neumorphism (soft shadows, low contrast), claymorphism (puffy 3D shapes, inner and outer shadows), Y2K and scrapbook (collage, stickers, playful type), brutalism, dark-mode dashboards
- [ ] Build one small page (a pricing card or login screen) in each style with Tailwind and plain CSS, so I learn the shadow, radius, gradient and spacing values behind each look
- [ ] Learn the rules that hold in every style: contrast ratios, a type scale, an 8-point spacing grid, consistent radius, accessibility. Neumorphism often fails contrast, so test it
- [ ] Collect layouts from galleries instead of starting blank: Dribbble, Behance, Mobbin (mobile app flows), Awwwards, Land-book, Godly. Keep a swipe file with a note on why each one works
- [ ] Rebuild 3 to 5 real layouts (hero, dashboard, pricing, settings) in React and Tailwind to train the eye and the hands
- [ ] Mobile: Flutter and React Native stay low priority. One weekend tutorial each (a to-do app) is enough to read the code; spend the real time on Expo, since I will already know React

Project A frontend: a web client (and optionally an Expo app) for the Pocket Plan API.

Phase 5: System design (weeks 10-24, then ongoing)

- [ ] Fundamentals: latency vs throughput, vertical/horizontal scaling, load balancers, caching layers and strategies, CDNs, replication/sharding/partitioning, CAP and consistency models, queues and event-driven design, rate limiting, API gateways, monolith vs microservices, idempotency, retries, observability
- [ ] Read: Designing Data-Intensive Applications (chapters 1 to 9 first), the System Design Primer on GitHub, ByteByteGo
- [ ] One design a week, written up in the repo: URL shortener, rate limiter, notification service, chat, news feed, payment system, file storage, search autocomplete, recommender serving
- [ ] Use a fixed framework: requirements, estimates, API, data model, high-level design, deep dive, bottlenecks, trade-offs
- [ ] Tie back to PDC: concurrency, MPI and Hadoop-style batch processing are the same ideas at scale

Phase 6: AI engineering (weeks 8-28, ongoing)

6.1 ML foundations refresh

- [ ] numpy, pandas, scikit-learn; train/test splits, cross-validation, precision/recall/F1/ROC, overfitting, regularization, feature engineering

- [ ] Data science habits: EDA, notebooks, clear plots

6.2 Deep learning

- [ ] PyTorch; rebuild the LSTM project properly. Stock prediction is an easy place to leak data and fool myself, so use walk-forward validation and compare against a naive baseline
- [ ] CNN, RNN/LSTM, transformers: Karpathy's Zero to Hero (build a tiny GPT), the Hugging Face course, basic fine-tuning (LoRA)

6.3 GenAI application development

- [ ] LLM APIs (Anthropic, OpenAI, Gemini): chat, streaming, structured outputs, tool/function calling
- [ ] Prompt engineering: clear instructions, examples, output schemas, system prompts, evals
- [ ] API key hygiene: never in client code or git, env vars or a secret manager, a server-side proxy, per-user rate limits, spend limits
- [ ] Cost and latency: token counting, caching, model routing
- [ ] Wrappers are a fine starting point; the real value is in evals, retrieval, guardrails and good UX

6.4 RAG

- [ ] Embeddings, chunking, vector search (pgvector, Qdrant or Chroma), hybrid search, reranking, citations, evaluating retrieval quality

6.5 Agentic AI

- [ ] Tool-use loops, planning, memory, MCP, multi-agent patterns, human-in-the-loop, guardrails, tracing (Langfuse or LangSmith)
- [ ] Use the vendor SDKs first, then LangGraph or similar
- [ ] AI coding tools (Claude Code, Codex): write a clear spec, let the tool plan, review diffs, run tests, commit small. Read every change; I should be able to explain every line (this matters in interviews)
- [ ] n8n: build 2 to 3 automations (webhook, LLM, database/email/Slack)

6.6 Recommender systems

- [ ] Popularity baselines, content-based, collaborative filtering, matrix factorization, implicit feedback, two-tower retrieval plus ranking, cold start
- [ ] Metrics: precision@k, recall@k, MAP, NDCG; offline vs online evaluation and A/B testing
- [ ] Dataset: MovieLens

6.7 MLOps and data platforms

- [ ] MLflow experiment tracking, model serving behind FastAPI, Docker, monitoring and drift basics

- [ ] Databricks (free edition): Spark DataFrames, Delta tables, notebooks, MLflow

6.8 Prompt and context engineering

- [ ] Prompt engineering: system vs user prompts, few-shot examples, structured outputs, reasoning trade-offs, prompt versioning
- [ ] Context engineering: what goes into the context window and in what order, trimming or summarising history, retrieval vs long context, keeping tool results small, caching the stable prefix
- [ ] Tokenization: BPE basics, why token counts differ for code and other languages, counting tokens before sending
- [ ] Chunking: fixed-size, recursive, semantic and structure-aware chunking; overlap; how chunk size changes retrieval quality
- [ ] Token and cost optimisation: shorter prompts, prompt caching, smaller models for easy steps, batching, output length caps, per-user budgets
- [ ] Evals: a small labelled test set, automated checks, LLM-as-judge with my own spot checks, a regression run on every prompt change, and tracking quality against cost and latency
- [ ] Write down the trade-offs for each project: bigger context vs retrieval, one big agent vs several small ones, speed vs accuracy, cost vs quality

6.9 Agent frameworks and tooling

- [ ] LLM APIs: practise with at least two providers; handle retries, timeouts, rate limits, streaming and errors
- [ ] LangChain for chains, retrievers and tool wrappers; LangGraph for stateful, branching agent workflows. Also build one agent with the raw SDK so I know what the frameworks hide
- [ ] Multi-agent workflows: planner and worker, router, reviewer or critic, parallel fan-out, and when a single agent is simpler and better
- [ ] Testing agents: unit-test the tools, mock model calls, replay recorded traces, write scenario tests with expected tool calls, and test guardrails against bad inputs
- [ ] MCP: write a small MCP server and connect it to a client; understand tools, resources, prompts and auth
- [ ] Claude Code workflow: project notes in CLAUDE.md, slash commands, subagents, hooks; Claude skills for repeatable tasks; Codex for comparison. Always review what the tool writes
- [ ] Vector databases: pgvector, Qdrant, Chroma, Pinecone; index types (HNSW, IVF), metadata filtering, hybrid search, and when plain Postgres is enough
- [ ] Wrapper to product: auth, per-user limits, logging with trace IDs, a feedback button, an admin view of failures, deployed with CI

6.10 Building models from scratch, computer vision, CUDA and a medical ML project

- [ ] ML algorithms: implement in numpy first (linear and logistic regression, k-means, k-NN, decision trees, naive Bayes), then use scikit-learn for the rest (SVM, random forests, gradient boosting with XGBoost or LightGBM)
- [ ] Train, evaluate and ship: train a model, save it, wrap it in a FastAPI service, containerise it and add monitoring (the MLOps loop from 6.7)
- [ ] Computer vision: OpenCV basics, image classification, object detection with a YOLO-style model, video streams. This connects directly to Toddle Safe's camera pipeline
- [ ] CUDA programming: GPU architecture, kernels, threads and blocks, the memory hierarchy, a matrix multiply and a reduction kernel. Work through the freeCodeCamp 12-hour CUDA course and put the repo on the resume; it builds on the PDC work
- [ ] Medical ML project: pick a public, de-identified dataset (for example chest X-rays or tabular patient-risk data), validate properly, explain the results (Grad-CAM or SHAP) and write a model card stating its limits. DeepLearning.AI's medical ML material is a good guide. Treat it as a learning project, not a diagnostic tool

6.11 Legal and compliance basics before shipping

This is general orientation, not legal advice. Before launching a real product with real users, have a lawyer review the documents for the relevant country.

- [ ] Data and privacy: collect only what is needed, get consent, define retention and deletion, encrypt storage, and learn which rules apply where users are (for example GDPR in the EU, CCPA in California). Health data has stricter rules (HIPAA in the US)
- [ ] Licences: open-source licences for code and dependencies (MIT, Apache 2.0, GPL), model licences and usage policies, dataset licences and terms
- [ ] Provider terms: read the API provider's usage policy and data-handling terms before building on it, and never send sensitive user data to a service that has not been checked
- [ ] User-facing documents: terms of service, privacy policy, cookie notice, and a clear AI disclosure (what is automated and what can be wrong)
- [ ] Responsible use: bias checks, a human fallback, no medical, legal or financial claims without review, logs kept for audits
- [ ] Payments: use Stripe's hosted checkout so card data never touches my servers, and learn refund and tax handling for each country I sell in

Project B: RAG + agent app (document Q&A or a support agent) with an eval set, tracing, deployed.

Project C: Recommender service on MovieLens with an offline evaluation report, an API and a simple UI.

Toddle Safe upgrade: write a proper architecture doc (camera ingest, detection models, alert service, reports), add diagrams and a demo video. This stays the headline project.

Phase 7: DevOps, cloud and testing (weeks 10-28, parallel)

- [ ] Linux admin basics, bash automation, systemd, SSH, networking tools
- [ ] Docker in depth (multi-stage builds, volumes, networks); Kubernetes concepts on a local cluster (kind or minikube): pods, deployments, services, config maps, ingress
- [ ] CI/CD: GitHub Actions pipelines for test, build, push image, deploy; dev/staging/prod environments as seen at the internship
- [ ] Infrastructure as code: Terraform basics
- [ ] One cloud properly. AWS has the widest market; Azure pairs naturally with Dynamics. For the consultancy route, start with Azure fundamentals (AZ-900) and deploy Project A there, then learn AWS basics
- [ ] Observability: logs, metrics, traces (Prometheus/Grafana or OpenTelemetry basics)
- [ ] Testing depth: unit, integration, e2e, TDD, mocks vs fakes, coverage, load testing (k6 or Locust), contract tests
- [ ] Project management habits: use GitHub Projects or Jira on my own work (sprints, tickets, retros)

Project management track (parallel, light)

I already studied agile, scrum, kanban, WBS and estimation. This track makes me use them on my own work, which is what interviewers and consultancy clients want to see.

- [ ] Revise: waterfall vs agile, scrum roles and events, kanban, themes, epics and stories, WBS, estimation (story points, planning poker), QA, risk registers
- [ ] Run my own projects as real sprints: one-week sprints, a backlog, a sprint goal, a review and retrospective notes, on GitHub Projects or Jira
- [ ] Metrics: velocity, burndown, cycle time, lead time, WIP limits (kanban), defect rate
- [ ] Documents: project charter, RACI, risk register, status report, release notes
- [ ] Estimation practice: estimate each task before starting, compare with the actual time, and note what I missed
- [ ] Consultancy angle: requirement workshops, fit-gap analysis, change requests, UAT sign-off (ties into Phase 8)
- [ ] Optional certifications: Scrum (PSM I or CSM), PMI CAPM, or an agile fundamentals course. Check current exam requirements before booking

Phase 8: Dynamics 365 and Power Platform track (parallel, 3 to 5 h/week from week 6)

- [ ] Power Apps: canvas (CareTrack) and model-driven apps; Dataverse tables, relationships, security roles

- [ ] Power Automate: cloud flows, approvals, connectors, error handling, custom connectors
- [ ] Copilot Studio (agents), which ties into Phase 6
- [ ] Dynamics 365 CE/CRM: sales and service customization, business rules, plug-in concepts
- [ ] D365 Finance and Operations: X++ basics, data entities, forms, classes, chain-of-command extensions, batch jobs, security, Visual Studio development, Lifecycle Services; get a trial or demo environment
- [ ] ERP functional basics (the accounting course helps): general ledger, AP/AR, procurement, inventory, fixed assets, budgeting
- [ ] C# and .NET basics
- [ ] Integrations: Dataverse Web API/OData, Azure Functions, Logic Apps, Entra ID auth
- [ ] Certifications to consider (check current exam codes and retirements on Microsoft Learn before booking): Power Platform Fundamentals (PL-900), Power Platform Functional Consultant (PL-200), and a D365 Finance functional or developer exam
- [ ] Consultancy skills: requirement gathering, BRD/FRD writing, fit-gap analysis, process mapping, UAT, training docs, stakeholder communication, estimation

Project D: CareTrack 2.0: a model-driven or improved canvas app, Dataverse, Power Automate approvals and a Power BI dashboard, documented as if for a client.

Phase 9: Data analytics and BI (weeks 14-30, light, parallel)

- [ ] Excel: pivot tables, Power Query, XLOOKUP, dashboards, charts, what-if analysis
- [ ] Power BI: data modelling (star schema), DAX basics, dashboards, publishing; Tableau on the same dataset to compare
- [ ] SQL for analytics: window functions, cohort and retention queries
- [ ] MATLAB (or free Octave): numerical methods (root finding, interpolation, ODE solvers, linear systems), matrix computation, plotting. Builds on the 4.0 in numerical computing and the differential equations course

Project E: analytics dashboard on a public dataset in both Power BI and Tableau, with a short insights write-up, plus a MATLAB numerical methods toolkit with visual demos.

Phase 10: Differentiators (once I am interviewing)

- [ ] Compiler construction: build a small interpreter (Crafting Interpreters) in TypeScript or Python
- [ ] Hands-on security: TryHackMe, OWASP Juice Shop, threat-model one of my own projects
- [ ] Quantum: see the Quantum computing track below
- [ ] Digital marketing: see the Digital marketing and growth section below
- [ ] More languages once two are solid: Java (Spring) or Go for backend roles, Rust for systems curiosity

- [ ] Parallel and distributed: Spark, Ray or CUDA mini projects building on the PDC work

Quantum computing track (light, after I start interviewing)

I studied the maths (Shor, Grover, QCNN, quantum ML). This track turns it into code and a portfolio piece.

- [ ] Refresh the maths: qubits, superposition, entanglement, gates (H, X, Z, CNOT), measurement, the Bloch sphere, tensor products. It is the linear algebra I already know
- [ ] Qiskit basics: build and simulate circuits locally, optionally run a small job on IBM Quantum's free access (check its current limits)
- [ ] Implement: a Bell state, Deutsch-Jozsa, Grover search on a small space, and Shor on a tiny number such as 15, with a clear note on what is simulated
- [ ] Quantum ML: a small variational classifier (PennyLane or Qiskit Machine Learning) on a toy dataset, and an honest comparison with a classical model
- [ ] Post-quantum cryptography: what Shor breaks (RSA, ECC), what holds up (larger symmetric keys, hash-based schemes), and NIST's post-quantum standards such as ML-KEM and ML-DSA. Link it to the information security work
- [ ] Hardware overview: superconducting, trapped-ion, photonic and neutral-atom qubits, noise, and why error correction matters
- [ ] Turn the quantum-and-encryption research paper into a clean GitHub repo: the PDF, the code, and a README with a one-page summary and diagrams. Add it to the resume only once the repo is public

Digital marketing and growth (light, parallel)

This makes me a product-minded engineer and helps if I ever consult or freelance.

- [ ] Revise from the course: SEO, SEM, SMO and SMM, e-commerce funnels, influencer marketing, A/B testing
- [ ] Build a portfolio site (Next.js) with proper SEO: titles, meta tags, sitemap, structured data (JSON-LD), Core Web Vitals, optimised images
- [ ] Analytics: Google Analytics 4 and Search Console, UTM tagging, conversion events, funnels
- [ ] A/B testing: sample size, significance, and not peeking. Run a real test on a landing page or behind a feature flag, and connect it to the recommender evaluation in 6.6
- [ ] Growth basics: landing page copy, email capture, a simple email sequence, a content calendar
- [ ] AI-assisted content: scripts, posts and short videos like the ones made in the course, with human review and a clear AI disclosure
- [ ] Product analytics for apps I ship (PostHog or similar): activation, retention and conversion

Resume plan

- [ ] One page, one column, ATS-friendly, no photos or skill bars
- [ ] Structure: header with GitHub/LinkedIn/portfolio, a 2-line summary, grouped and honest skills, experience, projects, education, certifications
- [ ] Write bullets as: did X, using Y, resulting in Z, with real numbers where I have them (120 unit tests, 6k lines, latency, users)
- [ ] Internship bullets: dev/staging/prod environments, express-validator validation, Postman API testing, time zone handling, Stripe documentation, code documentation, serverless CI/CD exposure. State accurately what I personally did
- [ ] Lead with Toddle Safe, then the SMTP server with tests, then Pocket Plan API, the RAG app, the recommender and CareTrack. Leave off projects a friend built
- [ ] Keep three versions: Backend/AI Software Engineer, AI Engineer, Dynamics/Power Platform consultant
- [ ] Refresh the resume at every milestone

Milestones

- [ ] M1 (end of week 5): comfortable in Python and TypeScript, 50 DSA problems, cleaned GitHub, SQL basics
- [ ] M2 (week 10): Project A API deployed with auth and Postgres, OS revision done, 100 DSA problems
- [ ] M3 (weeks 14-16): Project A frontend, Docker and CI, first system design write-ups. Start applying with resume v1
- [ ] M4 (weeks 20-24): Project B deployed, Stripe flow working, 150 DSA problems, mock interviews
- [ ] M5 (week 28+): Project C recommender, Project D CareTrack 2.0, Kubernetes and cloud basics, first certification

Weekly rhythm (adjust to my hours)

- Daily: 1 h DSA, 1 h core revision (OS, DB, networks, security), 3 to 4 h building
- Weekly: one system design write-up, one resume/LinkedIn/GitHub touch-up, one math or reading session, one review of the daily log
- Monthly: re-plan and drop whatever is not working

Video and course playlist

These are the ones I named, grouped by phase. One video or module a day alongside building. I have not checked exact titles, so use the names as search terms.

Phase 1: Python

- [ ] CS50 Python

Phases 3 and 5: backend and system design

- [ ] Caleb Curry: backend software engineer mind map, API design and architecture
- [ ] The backend-from-first-principles playlist
- [ ] Advanced backend: the 10 concepts every backend developer should know
- [ ] How to handle 1 million requests per second
- [ ] Stack overview videos: MERN, MEAN, PERN, T3, Java, Python, .NET
- [ ] Videos on request types, cloud computing, Docker deployment and ACID properties in SQL
- [ ] freeCodeCamp system design course

Phase 6: AI and the maths under it

- [ ] Andrej Karpathy's videos (start with Neural Networks: Zero to Hero)
- [ ] 3Blue1Brown's linear algebra, calculus and neural network series
- [ ] DeepLearning.AI short courses and specializations: LLMs, RAG, agents, MLOps, medical ML
- [ ] AI Engineer talks, for how people ship agents and RAG in production

CUDA

- [ ] freeCodeCamp 12-hour CUDA course (see 6.10)

Resources

- OSTEP (Operating Systems: Three Easy Pieces), NeetCode, Designing Data-Intensive Applications, System Design Primer, ByteByteGo
- Official docs for FastAPI, Express, Stripe, Docker and React; Anthropic, OpenAI and Gemini docs and cookbooks
- Karpathy's Neural Networks: Zero to Hero, the Hugging Face course, fast.ai
- Microsoft Learn (Power Platform, D365 F&O, Azure); Crafting Interpreters; TryHackMe

Open items to fill in

- Time available and any application deadline (this sets the pace)
- Toddle Safe: tech stack and what I built personally
- Internship: company, stack and what I shipped
- Current resume text
- Which languages I have written beyond C, C++ and Python (JS/TS from the internship)